

REDES NEURAIS ARTIFICIAIS

A tecnologia de redes neurais procura processar informações do mesmo modo como o cérebro humano o faz. Para tanto, utiliza processos e métodos nada convencionais. Lança a sua fundamentação fazendo analogias entre células nervosas vivas e o processo eletrônico, enfatizando o *aprendizado* como forma de captação de conhecimento. Conceitos como *aprender* e *treinamento* são uma constante do início ao final deste trabalho.

Graças a esse novo comportamento, computadores tem sido treinados para tarefas nas áreas de reconhecimento de padrões, previsões e controles diversos. Apesar da aplicação dessa tecnologia ser, ainda, um tanto tímida, possui, com certeza, um vasto campo de aplicações. Poder-se-ia citar, entre outras aplicações, o controle de qualidade, a previsão de preços de ações, o reconhecimento de vozes e imagens, a determinação de diagnósticos diversos, ...

Em "Redes Neurais Artificiais", Tafner, Xerez e Rodrigues Filho reúnem, de uma maneira bastante didática, os aspectos fundamentais para o entendimento e o uso de redes neurais.

Trata-se de uma obra que consegue reunir, desde aspectos teóricos básicos, até detalhes importantes de implementação de uma rede neural.

editora
EKO

EDITORIA DA
FURB

ISBN 85-7114-050-2



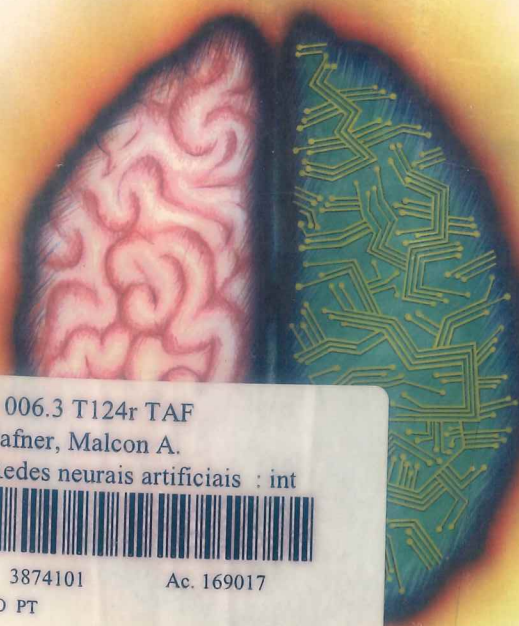
9 788571 140509

006.3
T124r
TAF

Ex.2

REDES NEURAIS INTRODUÇÃO E PRINCÍPIOS DE NEUROCOMPUTAÇÃO ARTIFICIAIS

Malcon A. Tafner, Marcos de Xerez e Ilson W. Rodrigues Filho



N.Cham. 006.3 T124r TAF
Autor: Tafner, Malcon A.
Título: Redes neurais artificiais : int



3874101

Ac. 169017

Ex.2 BCCO PT

editora
EKO

EDITORIA DA
FURB

Este livro
gem amp
redes neu
com uma
técnicas d
suas poss
no prese
Aborda,
influência
natureza
considera
em segui
nhos con
redes neu
sua criaçã
ressurgim
pois de c
descaso c
ciar uma
nos neur
uma peq
respeito d
seus neu
plexas es
leitor tem
pios, a fu
tetura da
então, é r
formação
cial, que s
neurônios
ral artific
ainda, sit
cópias.
projeto d
todas as
organizac
de seleçã
conjunto
efetuar o
Expõe a
proporçã
de uma i
cadêmica
orientada
neural, id
bustos, m
implemen
enta doi
redes neu
plos de t
nesmos.
o um e
e apren
eural im

EDITORA
EKO

CEFET/ES - BIBLIOTECA
UNED - Colatina

Registro nº 388
Data 08.02.00


EDITORA DA FURB

© Copyright Malcon A. Taffner, Marcos de Xerez e
Ilson W. Rodrigues Filho, 1995

Editora da FURB
Rua Antônio da Veiga, 140. Bloco T, sala 117.
89012-900 - Blumenau SC - BRASIL
Fones: (047) 321-0329 / 321-0330 - Fax: (047) 322-8818

EDITORA EKO
Rua Dr. Amadeu da Luz, 260 - 89010-160 - Blumenau - SC - Brasil
Fone: (047) 326-0499 - Fax: (047) 326-3062
E-mail: alemaeko@nutechnet.com.br

Capa e Editoração Eletrônica:
VANGUARDA PRODUÇÕES & MARKETING
Rua Visconde de Mauá, 54 - Fone: (047) 322-4590 - Blumenau - SC

Distribuição: Editora da FURB e Editora Eko

Depósito legal na Biblioteca Nacional, conforme decreto nº 1825,
de 20 de dezembro de 1907.

"Impresso no Brasil / Printed in Brazil"

FICHA CATALOGRÁFICA

Elaborada pela Biblioteca Central da FURB

Tafner, Malcon Anderson
T124r Redes neurais artificiais: introdução e princípios de neurocomputação /
Malcon A. Tafner, Marcos de Xerez e Ilson W. Rodrigues Filho. - Blumenau : EKO :
Ed. da FURB, c1995.
199p. : il.
ISBN 85-7324-030-X (EKO)
ISBN 85-7114-050-2 (FURB)
Inclui bibliografia e índice.
1. Redes neurais (Computação). 2. Inteligência artificial. I. Xerez, Marcos
de. II. Rodrigues Filho, Ilson Wilmar. III. Título.

CDD 006.3

CEFET/ES - BIBLIOTECA
UNED - Colatina

AGRADECIMENTOS

Professor Ricardo Miranda Bárcia, PhD

Professora Aurora Akiko Kawahara, Dra.

Eng. Marcos Teixeira Pochmann

Professor Olívio Pedron

Coordenação de Aperfeiçoamento de Pessoal

de Ensino Superior - CAPES

Universidade Regional de Blumenau - FURB

e

Universidade Federal de Santa Catarina

(Programa de Pós-graduação em Engenharia de Produção)

Dedicado

à

Marilyn Ítala Tafner

Sabemos que, para muitos casos, como para resolver a dificuldade de aprender alguns fatos, a solução pode estar no acréscimo de neurônios ou, ainda, de camadas escondidas. Todavia, como já foi dito anteriormente, deve se pensar bem antes de mexer na estrutura da rede, primeiro se esses fatos realmente adicionam o entendimento do problema como um todo, ou, então, se agora com tantos novos neurônios escondidos, a rede estará memorizando em vez de generalizando.

Bibliografia

- KOSKO, B. *Neural Networks for Signal Processing*. USA, Califórnia: University of Southern California, 1992.
- LAWRENCE, J. *Introduction to Neural Networks and Expert Systems*. USA, Califórnia: California Scientific Software, 1992.
- LIPPMANN, Richard. *An Introduction to Computing with Neural Works*. IEEE Computer Society, v. 3, n° 4, p. 4-22, abr. 1987.
- HINTON, Geoffrey. *Learning in Papallel Networks*. BYTE: Artificial Intelligence, USA, New York, v. 10, n° 4, p. 265-273, abr. 1985.
- MASTERS, Timothy. *Practical neural Networks recipes in C++*. USA, Califórnia: Academic Press, 1993.

6

PERCEPTRON

Introdução

O Perceptron foi criado em 1957 por Frank Rosenblatt. A partir desta data foi crescendo o interesse pelas redes neurais. O Perceptron, em sua origem, era uma simulação computacional para a retina, que demonstrou como o sistema nervoso visual reconhece padrões. Houve muito otimismo entre os pesquisadores, pois, aparentemente, parecia que os Perceptrons podiam resolver vários tipos de problemas. Iniciou-se, então, intensa pesquisa sobre redes neurais.

Marvin Minsky e Seymour Papert, porém, provaram, no livro intitulado "Perceptrons", que redes neurais de uma única camada, do tipo Perceptron, não seriam capazes de resolver problemas interessantes. O Perceptron, por exemplo, não pode resolver o problema XOR (ou exclusivo). Assim, o Perceptron foi criticado e acabou sendo apelidado de rede "brain-damaged", pois não poderia resolver nenhum problema realmente interessante. A pesquisa foi desestimulada.

Entretanto, não se extinguiu de todo. Pesquisando em silêncio (para talvez não cair no ridículo e evitar críticas de colegas), alguns, um pouco mais destemidos (ou temerosos!) continuavam trabalhando. E, em 1982, voltou a renascer o interesse pelas redes neurais, com a apresentação à National Academy of Sciences, por John Hopfield, de um trabalho sobre redes neurais (a rede que ele apresentou leva hoje o seu nome).

A rede neural Perceptron

A rede neural Perceptron é formada pelos seguintes elementos:

- Camada de entrada:** cada elemento da camada de entrada faz a distribuição do sinal que ele recebe para todas as unidades de processamento.
- Neurônios:** são constituídos de unidades sigma (Σ) e de funções de transferência (T). As unidades sigma fazem a soma ponderada dos sinais vindos pelas conexões com os elementos de entrada. As funções de transferência determinam a saída de cada neurônio (unidade de processamento) em função da soma ponderada.

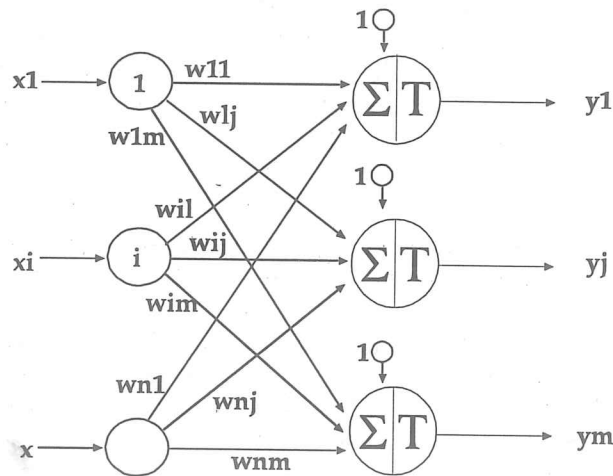


Fig. 53- Rede Neural Perceptron

Na rede apresentada pela figura acima temos:

$x_1, x_2, \dots, x_n$: sinais de entrada

w_i : peso da conexão entre a unidade de entrada i e a unidade sigma j , onde $i=[1,n]$ e $j=[1,m]$

s_j : soma ponderada da unidade sigma j

y_j : sinal de saída do neurônio j

As redes Perceptron têm apenas uma camada de processamento. Vem daí a limitação de representação por redes Perceptrons de apenas problemas linearmente separáveis. Não existe um procedimento para ajustamento de pesos em redes Perceptron com mais de uma camada (pelo menos não havia quando o Perceptron foi criado!). Estes problemas foram superados em modelos de redes que surgiram a partir dos anos 80 com o desenvolvimento de técnicas de aprendizado como, por exemplo, o *backpropagation*. Mas, estas redes receberam outros nomes. Deixaram de ser Perceptrons.

Neurônio (Unidade de Processamento)

O elemento de processamento ou unidade de processamento ou, ainda, neurônio do Perceptron pode ser representada da seguinte forma:

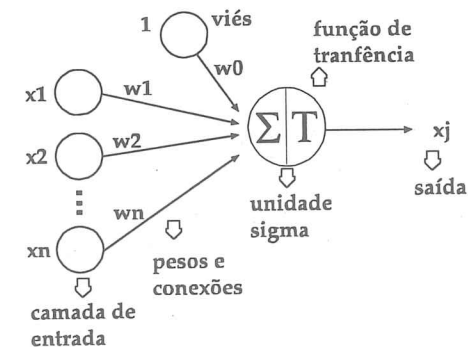


Fig. 54 - Unidade de processamento do Perceptron

Viés: é um elemento especial da camada de entrada, cujo sinal tem sempre valor 1. A conexão do viés com a unidade sigma tem pesos w_0 que são ajustados da mesma forma que os demais pesos.

Camada de entrada: a camada de entrada apenas conduz o sinal que chega ao Perceptron até sua unidade sigma.

Pesos e conexões: os pesos estão associados às conexões que levam os sinais vindos da camada de entrada até a unidade sigma e que simulam as eficácias das sinapses do neurônio biológico.

Unidade sigma: A unidade sigma faz a soma ponderada dos sinais de entrada. Ou seja, multiplica cada sinal de entrada pelo peso correspondente à conexão por onde passa o sinal, fazendo a soma total destes produtos.

Se x_i = sinais vindos de unidades i ou sinais vindos de pre-processadores, e

w_i = pesos associados às conexões da camada de entrada com a unidade sigma.

onde $x_i = \{x_0, x_1, x_2, \dots, x_n\}$ e
 $w_i = \{w_0, w_1, w_2, \dots, w_n\}$

então $s_j = x_0 w_0 + x_1 w_1 + x_2 w_2 + \dots + x_n w_n$ (1) e

$$s_j = \sum_{i=0}^n x_i w_i \quad (2)$$

Observação: O sinal de viés é sempre 1 (x_0). Já o w_0 é o peso da conexão entre o viés e a unidade sigma.

Função de transferência: a função de transferência vai determinar a saída do Perceptron conforme o valor que lhe for passado pela unidade sigma.

O valor de y_j (saída do Perceptron) será
$$\begin{cases} 0 \leftarrow s_j \leq 0 \\ 1 \leftarrow s_j > 0 \end{cases}$$

Classificação de padrões

Os Perceptrons são utilizados em problemas de classificação, ou seja, eles aprendem a classificar os padrões de entrada em grupos ou classe. A título de exemplo, digamos que vamos construir um Perceptron para reconhecer letras manuscritas. Assim, para cada padrão de entrada correspondente a uma letra "A" manuscrita, ele deve classificar este padrão na classe "A" de letras. As letras A, manuscritas, devem ser reconhecidas como sendo da mesma classe.

Seja uma rede Perceptron de uma única camada de processamento. A unidade sigma vai fazer a seguinte soma ponderada:

$$s_j = x_0 w_0 + x_1 w_1 + x_2 w_2 + \dots + x_n w_n$$

Logo:
$$s_j = \sum_{i=0}^n x_i w_i$$

Vamos supor uma camada de entrada com apenas dois elementos, x_1 e x_2 .

s_j será dado por $s_j = x_0 w_0 + x_1 w_1 + x_2 w_2$ e,

como $x_0 = 1$, temos, então, $s_j = w_0 + x_1 w_1 + x_2 w_2$

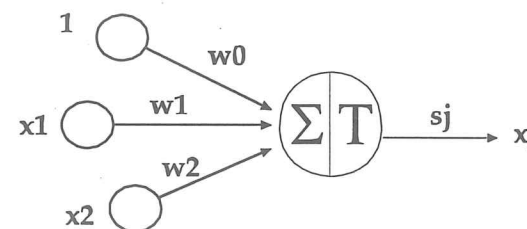
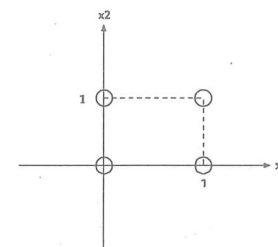


Fig. 55 - Rede Perceptron de apenas duas entradas

A saída para y_j é
$$\begin{cases} 0 \leftarrow s_j \leq 0 \\ 1 \leftarrow s_j > 0 \end{cases}$$

Desde que x_1, x_2 podem apresentar sinais 0 ou 1, podemos representá-los graficamente, e, assim teremos:



Fazendo $s_j=0$, temos pela equação (1) :

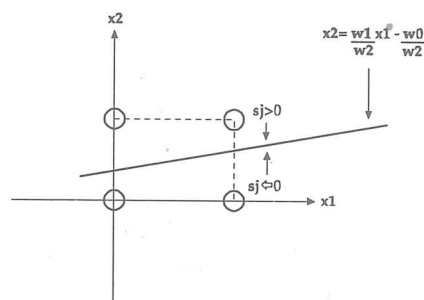
$$w_0 + x_1 w_1 + x_2 w_2 = 0;$$

portanto,

$$x_2 w_2 = -x_1 w_1 - w_0$$

$$x_2 = \frac{-w_1 x_1}{w_2} - \frac{w_0}{w_2}$$

Esta é uma equação de reta



A reta representada por esta equação separa as saídas do Perceptron em duas classes: a classe $y_i=0$ para $s_j \leq 0$ e $y_i=1$ para $s_j > 0$.

A separação entre essas duas classes é chamada de superfície de decisão. Como existem apenas duas entradas (x_1 e x_2), esta superfície de decisão é uma reta. Quando existirem mais do que duas entradas conectadas com a unidade sigma, vamos ter um hiperplano.

Um Perceptron de uma única camada com duas unidades de entrada e um único neurônio (unidade de processamento) só pode resolver problemas de classificação quando for possível separar as duas classes possíveis dos padrões de entrada, por uma reta. Ou seja, as classes devem ser linearmente separáveis.

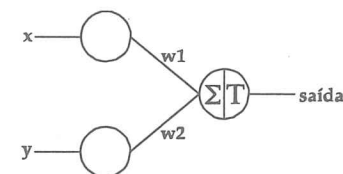
QUADRO 8- O problema do XOR (ou-exclusivo)

Diz-se, assim, que o Perceptron só pode resolver classes de problemas linearmente separáveis.

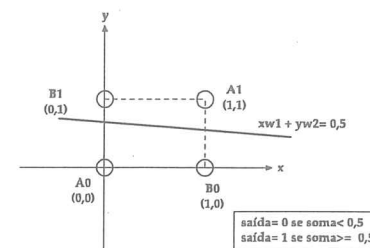
Um dos resultados desencorajadores de Minsky mostra que um Perceptron de camada única não pode simular uma simples função ou-exclusivo (XOR). Esta função, ou-exclusivo, é representada pela tabela abaixo, onde X e Y representam as duas entradas e S a saída:

X	Y	S
0	0	0
0	1	1
1	0	1
1	1	0

Esta função aceita duas entradas que somente podem ser 0 ou 1, e produz uma única saída também sendo 0 ou 1. Segue abaixo o sistema:



O plano abaixo representa x e y, sendo as duas entradas, e todas as combinações possíveis (conforme a tabela já mostrada):



A função de transferência desta rede neural, deve produzir 0 para saída quando a soma for menor que 0,5, e produzir 1 quando soma for maior ou igual a 0,5.

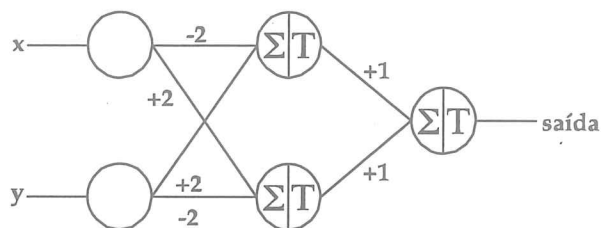
O neurônio executa o seguinte cálculo de soma ponderada: $soma = xw_1 + yw_2$

Os valores de entrada em um lado da reta produzirão soma $> 0,5$, logo, a saída será igual a 1. Os valores do outro lado da reta produzirão a soma $< 0,5$ onde, então, a saída será igual a 0. Outro par, w_1 e w_2 , mudará a inclinação e posição da reta. Verificando abaixo todas as combinações e seus resultados

Se $x=0$ e $y=0$, então	$soma=0$,	logo	saída= 0
Se $x=1$ e $y=0$, então	$soma=w_1$,	logo $w_1 \geq 0,5$	saída= 1
Se $x=0$ e $y=1$, então	$soma=w_2$,	logo $w_2 \geq 0,5$	saída= 1
Se $x=1$ e $y=1$, então	$soma=w_1+w_2$	logo $w_1+w_2 \geq 5$	saída deverá ser 1 e não 0

verificamos que, não é possível que a tabela XOR seja reproduzida por um Perceptron.

Uma forma clássica de resolver esse problema é descaracterizar a rede adicionando mais uma camada de neurônios. Uma camada escondida. Neste caso, cada neurônio terá um limite de 1, significando que o neurônio disparará somente se a soma das entradas totalizar 1, ou mais. Com essa nova arquitetura, que já não lembra mais o Perceptron, é possível realizar a tabela XOR com os seguintes pesos:



Executando a rede neural acima obtemos os seguintes resultados:

Entradas	X	Y	Saída
	0	0	0
	0	1	1
	1	0	1
	1	1	0

Aprendizagem do Perceptron

A aprendizagem de uma rede neural consiste na apresentação, a ela mesma, de um conjunto de padrões correspondente ao objeto que ele deve reconhecer, ajustando-se os pesos das conexões entre a camada de entradas com a unidade sigma, para obter a saída desejada. Ao ajustamento de pesos denominamos de **treinamento**.

Dizemos que a aprendizagem do Perceptron é do tipo supervisionada porque requer que o comportamento da rede durante o treinamento seja avaliado para se decidir o que fazer para que os pesos seja devidamente ajustados, conforme veremos a seguir.

Vamos supor que queiramos treinar uma rede Perceptron simples (apenas uma unidade de treinamento) para reconhecer as letras "A" e "B" (manuscritas). Como vamos ter apenas um neurônio, este só poderá ser utilizada para classificar os sinais de entrada em dois grupos, já que as saídas possíveis só podem ser 0 e 1. Se quiséssemos classificar os sinais de entrada numa quantidade maior de grupos, a rede deverá ter, necessariamente, mais neurônios (já que cada unidade dá origem a um sinal de saída y_i). Se necessitarmos classificar sinais de entrada em 4 grupos, por exemplo, vamos precisar de pelo menos dois neurônios na rede. A codificação para estes 4 grupos seria, então: (0,0), (0,1), (1,0) e (1,1). Os sinais y_1 e y_2 representam, respectivamente:

y_1 : sinal de saída do 1º neurônio
 y_2 : sinal de saída do 2º neurônio

Conclui-se, assim, que o número de grupos que se pode classificar com uma rede de Perceptrons é de 2^n grupos, onde n é o número de neurônios da rede (unidades de processamento).

Pré-processamento

Algum tipo de pré-processamento vai ser necessário para que se possa transformar os padrões de letras manuscritas "A" e "B" num padrão de sinais que possa ser entendido pelo Perceptron.

Poderíamos, por exemplo, transformar a letra num vetor de dígitos 0 e 1, da seguinte forma:

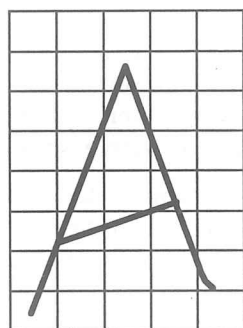


Fig. 56 - Letra "A"

Uma vez desenhando o sinal no tabuleiro, a transformação da informação não é difícil, bastando apenas identificar os pontos que apresentarem uma significância razoável de preenchimento no quadrado. Depois de identificados, basta apenas seguir o passo 2, descrito acima, e atribuir os sinais 0 e 1 para os preenchimentos respectivos.

- (1) Desenhamos um tabuleiro com $R \times S$ quadrados, onde vamos traçar uma letra ("A" ou "B").
- (2) Cada quadrado do tabuleiro marcado pelo traçado da letra será representado por 1. Cada quadrado vazio, ou seja, que não foi marcado pelo traçado da letra, será representado por 0.
- (3) Vamos representar os quadrados 1 e 0 como elementos de um vetor, obtendo-se uma matriz de 0s e 1s.

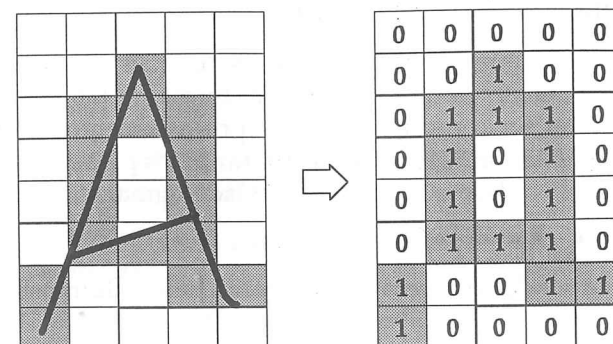


Fig. 57 - Conversão de sinais

O vetor obtido como resultado dessa conversão a partir da matriz da figura 57, poderia ser visto da seguinte forma:

$$\underbrace{[0,0,0,0,0]}_{1 \text{ linha}}, \underbrace{[0,0,1,0,0]}_{2 \text{ linha}}, \underbrace{[0,1,1,0,0]}_{3 \text{ linha}}, \dots, \underbrace{[1,0,0,0,0]}_{n \text{ linha}}$$

O vetor de sinais de entrada x , do Perceptron, a ser apresentado será:

$$x = [1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 0, \dots, 1, 0, 0, 0, 0]$$

onde o primeiro elemento do vetor é 1, sendo ele o sinal de viés (o sinal do viés é sempre 1).

Cada elemento deste vetor, ou seja, $x_0, x_1, x_2, \dots, x_n$, vai ser multiplicado pelo peso correspondente da conexão entre a entrada com a unidade sigma, obtendo-se, assim, a soma ponderada S :

$$S = x_0 w_0 + x_1 w_1 + x_2 w_2 + \dots + x_n w_n \quad \text{onde } x_0 = 1.$$

Os pesos iniciais das conexões vão ser arbitrários. Poderíamos começar o treinamento igualando-os todos a zero, por exemplo.

O nosso Perceptron de um neurônio vai ter duas saídas: 0 e 1. Vamos arbitrar que a saída 0 represente a letra "B" e a saída 1 represente a letra A (poderíamos arbitrar ao contrário, sem problema nenhum).

Treinamento do Perceptron

Vamos aplicar um padrão qualquer X de sinais relativos a uma letra "A" manuscrita no diagrama em forma de tabuleiro. Esse padrão X vai ser o resultado do pré-processamento, conforme discutimos anteriormente. Vamos arbitrar pesos para as conexões. A soma ponderada, executada pela unidade sigma, será:

$$S = x \cdot w = x_0 w_0 + x_1 w_1 + x_2 w_2 + \dots + x_n w_n.$$

A soma ponderada S será passada para a função de transferência,

$$\text{se } \begin{cases} S > 0 \therefore y_j = 1 \\ S \leq 0 \therefore y_j = 0 \end{cases}$$

e y_j será a saída da unidade de processamento.

Se a saída y_j for 1, estará correta ("A" é representado pela saída 1 conforme convencionamos). Nada se faz com os pesos e apresentamos outra letra ao Perceptron.

Se, ao contrário, a resposta for 0, ao invés de 1, podemos concluir intuitivamente que se pode aumentar o valor de S para torná-lo positivo (se $y_j = 0$ significa que a soma $S \leq 0$). Como não se pode mexer nos sinais X , pois eles representam a letra (e são resultantes do pré-processamento), quer-nos parecer que, aumentando os pesos, conseguiríamos aumentar o valor de S , tornando-o positivo. É claro que não estamos entrando na consideração dos sinais dos pesos.

De maneira análoga, apresentamos uma letra "B" ao Perceptron. Se a saída y_j for 0, não se faz nada com os pesos, pois a saída está correta. Mas, se a saída y_j for 1, significa que a soma $S > 0$. Como S precisa ser ≤ 0 , intuitivamente vamos baixar os pesos para a soma ponderada se tornar negativa ou nula.

Desta forma, determinamos o método de treinamento do Perceptron, que pode, então, ser resumido no seguinte algoritmo:

Algoritmo de Treinamento do Perceptron

- 1º passo: arbitrar pesos para as conexões entre a camada de entrada e a unidade sigma;
- 2º passo: aplicar um padrão de sinais de entrada x_j e calcular a soma ponderada S ;
- 3º passo: passar a soma ponderada S para a função de transferência:
 - a) se a saída y_j estiver correta, voltar ao 2º passo
 - b) se a saída y_j estiver errada e for 0, adicionar cada peso das conexões com os sinais de entrada relativos a elas,
 - c) se a saída y_j estiver errada e for 1, subtrair de cada peso das conexões os sinais de entrada relativos a elas;
- 4º passo: voltar ao 2º passo.

Um exemplo de verdade com números de verdade

Para um melhor entendimento da rede neural Perceptron, vamos desenvolver um pequeno exemplo de aprendizado. O exemplo será composto por poucos elementos para simplificar a demonstração. No nosso exemplo desejamos que a rede aprenda a distinguir ídolos do esporte nacional. Vamos treinar a rede para reconhecer dois grandes jogadores de futebol (Pelé e Zico) e dois grandes pilotos de automobilismo (Emerson e Piquet).

	Futebol	Automobilismo
Pelé	x	
Zico	x	
Emerson		x
Piquet		x

Isso quer dizer que, quando a rede estiver treinada, poderemos apresentar qualquer um dos ídolos acima à rede e a mesma deverá identificar qual a atividade exercida pelo personagem apresentado.

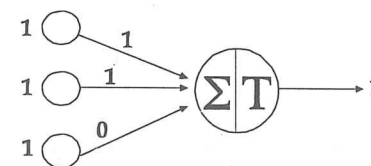
Para tanto, antes de prosseguir com a rede neural, sabemos da necessidade de codificar a informação, tanto para as entradas quanto para as saídas desejadas. Essa necessidade parte do princípio que em uma rede neural tipo Perceptron, os valores devem assumir condições binárias, ou seja, 1 ou 0. Neste caso, uma proposta de identificação dos conjuntos da tabela acima pode ser codificada da seguinte forma:

		0	1
		Futebol	Automobilismo
00	Pelé	x	
01	Zico	x	
10	Emerson		x
11	Piquet		x

Acrescentando o VIÉS (1) ao conjunto de entrada temos, então, o conjunto com o seguinte aspecto:

100	Pelé
101	Zico
110	Emerson
111	Piquet

Podemos, agora, traçar o desenho da rede neural baseando-nos nas tabelas de dados (conjunto de treinamento). Pelas tabelas acima, podemos notar que a saída se constitui de apenas duas posições, 1 (0) ou automobilismo (1), logo, podemos usar um neurônio de saída que, identificando 100 (Pelé) ou 101 (Zico) produza 0 (futebol), e, identificando 110 (Emerson) ou 111 (Piquet) produza 1 (automobilismo) como saída. Para as entradas usaremos 3 elementos de entrada, um para o viés (1) e os outros dois para as informações relativas aos ídolos (00, 01, 10 e 11).



Iniciando os pesos do neurônio com 0 temos o seguinte conjunto de pesos:

$$w = \{w_0, w_1, w_2\} = \{0, 0, 0\}$$

A função de transferência do neurônio é igual a:

se soma ponderada > 0 então saída = 1
senão saída = 0

O conjunto de treinamento constitui os seguintes pares:

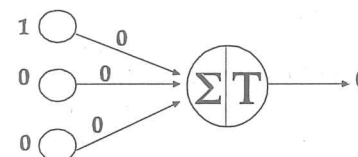
entrada 1 = $\{x_0, x_1, x_2\} = \{1, 0, 0\}$, cuja saída é saída 1 = $\{y\} = \{0\}$

entrada 2 = $\{x_0, x_1, x_2\} = \{1, 0, 1\}$, cuja saída é saída 2 = $\{y\} = \{0\}$

entrada 3 = $\{x_0, x_1, x_2\} = \{1, 1, 0\}$, cuja saída é saída 3 = $\{y\} = \{1\}$

entrada 4 = $\{x_0, x_1, x_2\} = \{1, 1, 1\}$, cuja saída é saída 4 = $\{y\} = \{1\}$

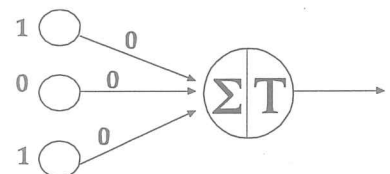
Apresentando pela 1ª vez a entrada 1 (Pelé):



Calculando a saída para a entrada 1
 $\sum x.w = x_0w_0 + x_1w_1 + x_2w_2$

Soma = $1.0 + 0.0 + 0.0 = 0$ e $FT(0) = 0$
saída = 0 (futebol) e
saída desejada = 0 (futebol), logo, a
saída está correta

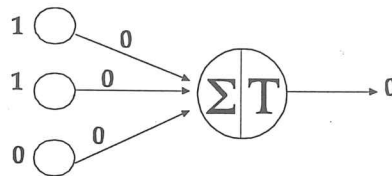
Apresentando pela 1ª vez a entrada 2 (Zico):



Calculando a saída para a entrada 2
 $\sum x.w = x_0w_0 + x_1w_1 + x_2w_2$

Soma = $1.0 + 0.0 + 1.0 = 0$ e $FT(0) = 0$
saída = 0 (futebol) e
saída desejada = 0 (futebol), logo, a
saída está correta

Apresentando pela 1ª vez a entrada 3 (Emerson):



Calculando a saída para a entrada 3
 $\Sigma x.w = x_0w_0 + x_1w_1 + x_2w_2$

Soma = $1.0 + 1.0 + 0.0 = 0$ e $FT(0) = 0$
 saída = 0 (futebol) e
 saída desejada = 1 (automobilismo),
 logo, a saída não está correta

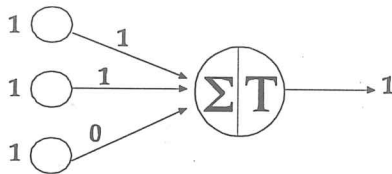
Como saída = 0 vamos adicionar a cada peso a entrada correspondente:

$$w_0 = 0 + 1 = 1$$

$$w_1 = 0 + 1 = 1$$

$$w_2 = 0 + 0 = 0$$

Apresentando pela 1ª vez a entrada 4 (Piquet):



Calculando a saída para a entrada 4
 $\Sigma x.w = x_0w_0 + x_1w_1 + x_2w_2$

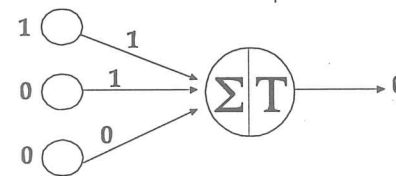
Soma = $1.1 + 1.1 + 1.0 = 2$ e $FT(2) = 1$
 saída = 1 (automobilismo) e
 saída desejada = 1 (automobilismo),
 logo, a saída está correta

Agora que todas as entradas do conjunto de treinamento foram apresentadas, deve-se fazer uma avaliação do treinamento. Essa avaliação poderá ajudar a decidir se a rede está ou não devidamente treinada, ou seja, ela reconhece ou não todos os conjuntos apresentados. Uma forma de avaliação é fazer um teste total, ou seja, apresentar todo o conjunto de treinamento e quando uma entrada não for reconhecida, ainda durante a apresentação do conjunto, interromper o teste, ajustar os pesos e continuar o treinamento. Outra proposta é realizar um teste por amostragem, quando o conjunto for muito grande. Quer dizer, fazer um teste com 10%, ou 20% (como o leitor preferir), do conjunto de treinamento e verificar o seu sucesso.

Outro ponto importante que deve ser levado em consideração é a precisão da rede. É certo que, dependendo do conjunto de treinamento e da arquitetura da rede neural, talvez ela não reconheça 100% dos casos apresentados à ela, mas reconhece uma percentagem razoável. Essa decisão deve ser tomada em tempo de treinamento.

Para o nosso caso em particular, podemos decidir pelo treinamento total, ou seja, 100% de reconhecimento, visto que o conjunto e a rede são pequenos e simples. Vamos, então, reiniciar o treinamento já que houve, em algum momento do treinamento anterior, um reajuste de pesos. Isso quer dizer que em algum momento, alguma entrada não foi reconhecida apropriadamente, e, dessa forma, vamos repassar o conjunto de treinamento para eliminar a possibilidade da rede não reconhecer alguma entrada do conjunto apresentado.

Apresentando pela 2ª vez a entrada 1 (Pelé):



Calculando a saída para a entrada 1
 $\Sigma x.w = x_0w_0 + x_1w_1 + x_2w_2$

Soma = $1.1 + 0.1 + 0.0 = 1$ e $FT(1) = 1$
 saída = 1 (automobilismo) e
 saída desejada = 0 (futebol), logo, a saída não está correta

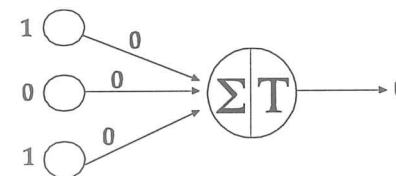
Como saída = 1 vamos subtrair de cada peso a entrada correspondente:

$$w_0 = 1 - 1 = 0$$

$$w_1 = 1 - 0 = 1$$

$$w_2 = 0 - 0 = 0$$

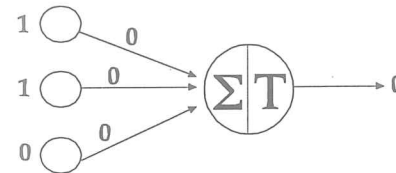
Apresentando pela 2ª vez a entrada 2 (Zico):



Calculando a saída para a entrada 2
 $\Sigma x.w = x_0w_0 + x_1w_1 + x_2w_2$

Soma = $1.0 + 0.1 + 1.0 = 0$ e $FT(0) = 0$
 saída = 0 (futebol) e
 saída desejada = 0 (futebol), logo, a saída está correta

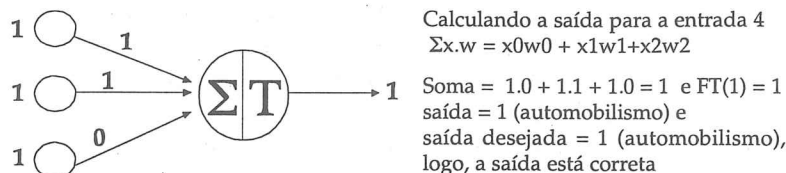
Apresentando pela 2ª vez a entrada 3 (Emerson):



Calculando a saída para a entrada 3
 $\Sigma x.w = x_0w_0 + x_1w_1 + x_2w_2$

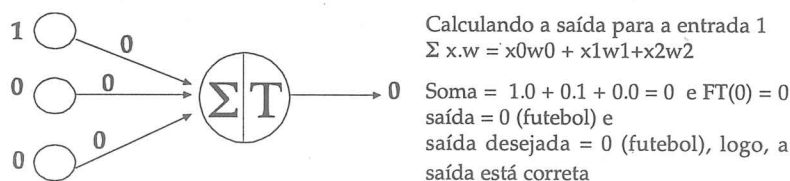
Soma = $1.0 + 1.1 + 0.0 = 1$ e $FT(1) = 1$
 saída = 1 (automobilismo) e
 saída desejada = 1 (automobilismo),
 logo, a saída está correta

Apresentando pela 2ª vez a entrada 4 (Piquet):



Conforme houve um ajuste de pesos logo no início deste segundo treinamento, poderíamos reiniciar pela 3ª vez todo o treinamento da rede neural. Entretanto, podemos dispensar todo este retreino vendo que as entradas 2, 3 e 4 produzem saídas corretas mediante os pesos { 0, 1, 0 } da rede. Precisamos apenas verificar se a saída produzida para a entrada 1 está correta, pois não necessariamente ajustando os pesos uma vez fará com que se produza a saída esperada.

Reapresentando a entrada 1 (Pelé):



Conforme o teste realizado acima, Pelé também foi reconhecido, o que significa que a rede reconheceu todos os casos apresentados do conjunto de treinamento.

Caso houvesse o interesse de que a saída fosse composta por mais de um bit (0 ou 1), por exemplo, 3 bits, bastaria acrescentar à ela o mesmo número de neurônios de saída. Isto significa que cada neurônio de saída corresponde a um bit de saída, e, que combinado com os outros tantos bits produzidos na mesma camada (camada de saída), acaba representando uma informação específica.

Para sermos mais precisos, podemos fazer uma pequena modificação no nosso exemplo e demonstrar como isso poderia ser realizado. Imaginemos que a saída será composta por mais de um bit, por exemplo 2 bits, teríamos, então, uma combinação diferente, onde cada neurônio representaria um esporte, assim, o neurônio 0 (y_0) representaria o futebol, e o neurônio 1 (y_1) representaria o automobilismo.

Diante desta composição o conjunto de treinamento ficaria com o seguinte formato:

entrada 1 = $\{x_0, x_1, x_2\} = \{1, 0, 0\}$, cuja saída é saída 1 = $\{y_0, y_1\} = \{1, 0\}$
 entrada 2 = $\{x_0, x_1, x_2\} = \{1, 0, 1\}$, cuja saída é saída 2 = $\{y_0, y_1\} = \{1, 0\}$
 entrada 3 = $\{x_0, x_1, x_2\} = \{1, 1, 0\}$, cuja saída é saída 3 = $\{y_0, y_1\} = \{0, 1\}$
 entrada 4 = $\{x_0, x_1, x_2\} = \{1, 1, 1\}$, cuja saída é saída 4 = $\{y_0, y_1\} = \{0, 1\}$

A arquitetura desta rede neural seria de 3 elementos na camada

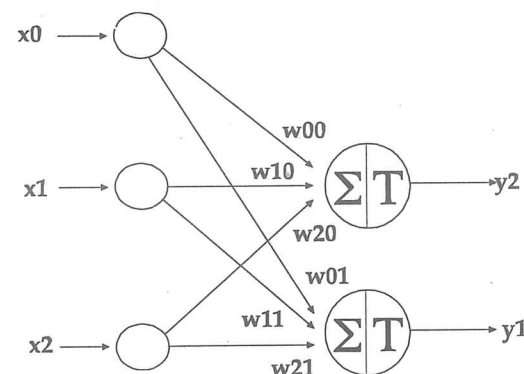


Fig 58 - Exemplo de uma rede com duas saídas

de entrada e dois neurônios de saída, todos totalmente conectados e com pesos iniciais aleatórios, ou com pesos igual a 0:

Bibliografia

- ALEKSANDER, I., MORTON H. .An Introduction to Neural Computing. Inglaterra, Londres: Chapman & Hall, 1990.
- ARARIBÓRIA, G. Inteligência Artificial. Rio de Janeiro: Livros Técnicos e Científicos, 1988.
- DAYHOFF, Judith. Neural Network Architectures. USA, New Yor: Van Nostrand Reinhold, 1990.
- HINTON, Geoffrey. Learning in Papallel Networks. BYTE: Artificial Intelligence, USA, New York, v. 10, n° 4, p.265-273, abr. 1985.